Integración por Monte Carlo en memoria compartida con OpenMP

Evaluación de programación paralela · Introducción a la computación paralela

Universidad Industrial de Santander · Grupo 28661 · Docente: Sergio Augusto Gélvez Cortés

Estudiante: **Nancy Liliana Saenz Moreno** · Código **2224510** · Semilla usada: **2224510**

Fecha de entrega: 27 de octubre de 2026

Contenido

- Parte I. Fundamento del método
- Parte II. Implementación en OpenMP
- Parte III. Evaluación de desempeño: speedup y ley de Amdahl
- Parte IV. Informe sobre su proceso de solución (con la bitácora de IA como anexo)
- Anexo. Programa paralelo completo (montecarlo.c)

Todos los programas se ejecutaron en mi portátil (AMD Ryzen 5 3450U, 4 núcleos y 8 hilos, Ubuntu en WSL, gcc 15.2.0). Las capturas de pantalla son de esas ejecuciones, y los programas, scripts y salidas completas están en el repositorio Git del proyecto.

Parte I. Fundamento del método

La idea en una imagen. Se lanzan puntos al azar sobre un cuadrado de lado 1, como si fueran dardos. La fracción que cae dentro del cuarto de círculo es $\pi/4$, así que $\pi \approx 4 \times$ (puntos dentro) $\div$ (puntos totales). Con los primeros 2000 puntos de mi semilla cayeron 1568 dentro, y eso da $\pi \approx 3{,}136$ (Figura I-1). Con 100 millones de puntos da $\pi \approx 3{,}141668$.

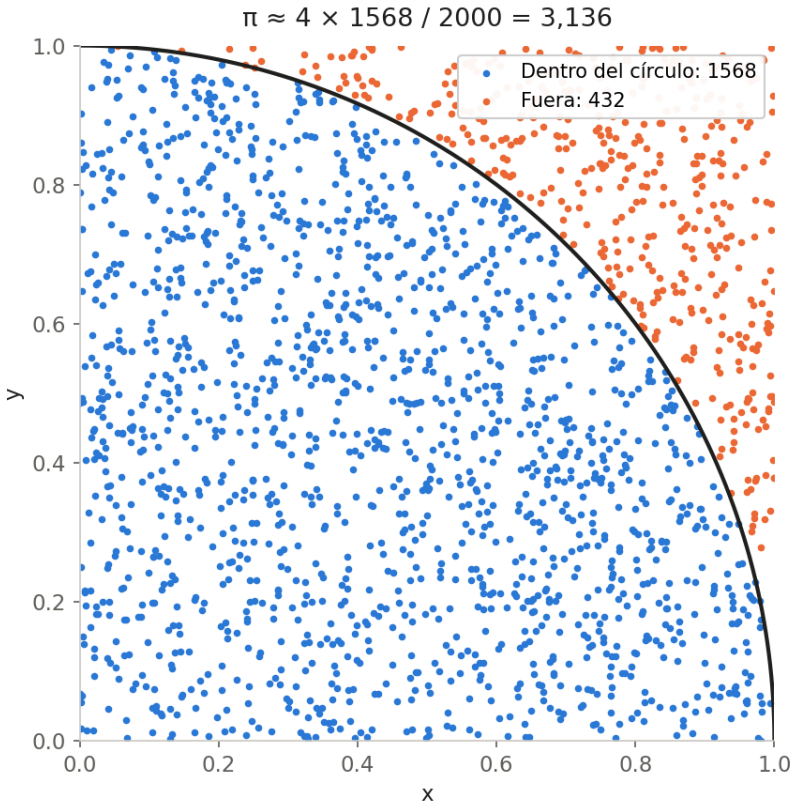


Figura I-1. Los primeros 2000 puntos generados con mi semilla (2224510): en azul los que caen dentro del cuarto de círculo y en naranja los que caen fuera.

I.1. Explique con sus propias palabras por qué el error del estimador disminuye en proporción a $1/\sqrt{N}$ y deduzca cuántas muestras adicionales se requieren para reducir el error a la mitad. ¿Qué implicación tiene esto sobre el costo de ganar un dígito de precisión?

¿Por qué $1/\sqrt{N}$? Cada punto es como lanzar una moneda cargada: cae dentro con probabilidad $q = \pi/4 \approx 0{,}785$ o cae fuera. Un solo punto no dice casi nada, pero al promediar muchos, los "errores de suerte" de unos se compensan con los de otros. Esa compensación no es perfecta: cuando se juntan N cosas al azar, la dispersión del promedio baja como $\sqrt{N}$, no como N. Por eso el error típico es

$$\text{error} \approx 4 \cdot \sqrt{(\, q(1-q) / N\,)} \approx 1{,}64 / \sqrt{N}.$$

Lo comprobé en mi computador (Figura I-2). Para cada N repetí el cálculo 30 veces, cada vez con un tramo distinto de números aleatorios, y promedié el error. Los puntos azules (mis datos) caen

casi encima de la línea naranja (la fórmula). Por ejemplo, con $N = 10^6$ el error medio fue 0,00175 y la fórmula predice 0,00164. Una sola corrida puede tener más o menos suerte (puntos grises): con $N = 10^6$ la mía salió con un error de apenas 0,00004. Por eso la regla habla del error **típico**, no del de cada corrida.

¿Cuántas muestras más para reducir el error a la mitad? Si el error es $1{,}64/\sqrt{N}$, para que sea la mitad hay que multiplicar $\sqrt{N}$ por 2, o sea **multiplicar N por 4**. Hay que agregar **3N muestras más** de las que ya se tenían. Con $N = 10^8$, para bajar el error de 0,000164 a 0,000082 se necesitan 4×10^8 puntos.

¿Qué implica para ganar un dígito? Un dígito más es un error 10 veces menor, y eso exige **100 veces más muestras** ($10^2 = 100$), es decir, 100 veces más tiempo de cálculo. Si 10^8 puntos tardan cerca de 1,3 s en mi portátil, ganar un dígito costaría unos 2 minutos, y otro dígito más, unas 3,6 horas. Monte Carlo es muy fácil de programar, pero **cada dígito de precisión sale cada vez más caro**.

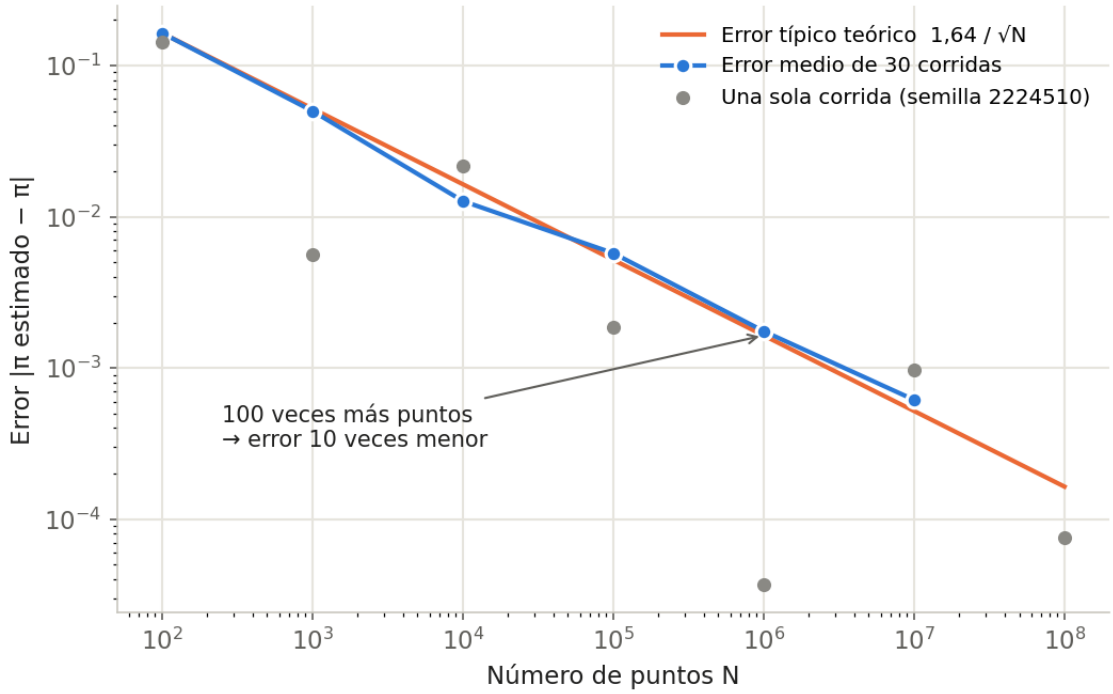


Figura I-2. Error frente al número de puntos (escala logarítmica). La línea naranja es la fórmula $1{,}64/\sqrt{N}$, los puntos azules son el promedio de 30 corridas y los grises, una sola corrida con mi semilla.

I.2. Justifique por qué la integración por Monte Carlo se clasifica como un problema embarazosamente paralelo: analice las dependencias entre las muestras (o su ausencia) y determine qué datos deben ser compartidos y cuáles deben ser privados cuando el muestreo se reparte entre varios hilos de ejecución.

¿Por qué "embarazosamente paralelo"? Porque se puede repartir casi sin esfuerzo: **ningún punto depende de otro**. Para saber si el punto número 500 cae dentro del círculo no hace falta conocer el punto 499 ni ningún otro. No hay que esperar resultados ajenos ni intercambiar datos mientras se calcula. Lo único que se junta es el **número de aciertos**, y eso se hace una sola vez, al final, sumando lo que contó cada hilo.

¿Qué se comparte y qué es privado? Aplicando lo visto en clase (lo declarado fuera de la región paralela es compartido y lo declarado dentro es privado), queda así (Figura I-3):

Dato	¿Compartido o privado?	Por qué
N (total de puntos)	Compartido	Todos lo leen y nadie lo cambia
Semilla base	Compartida	Solo se lee, para calcular dónde empieza cada hilo
Estado del generador	**Privado**	Si fuera compartido, los hilos se pisarían al pedir números (condición de carrera)
x, y del punto	Privados	Cada hilo calcula sus propios puntos
Variable del ciclo	Privada	OpenMP la hace privada automáticamente
Contador de aciertos	Privado mientras se cuenta y **sumado al final**	Se hace con reduction(+:dentro): cada hilo cuenta en su copia y OpenMP suma las copias

Hay que cuidar dos cosas. El contador no puede ser una sola variable compartida, porque varios hilos sumarían a la vez y se perderían aciertos. Y el generador de números aleatorios tiene que ser propio de cada hilo y no repetir los números de otro.

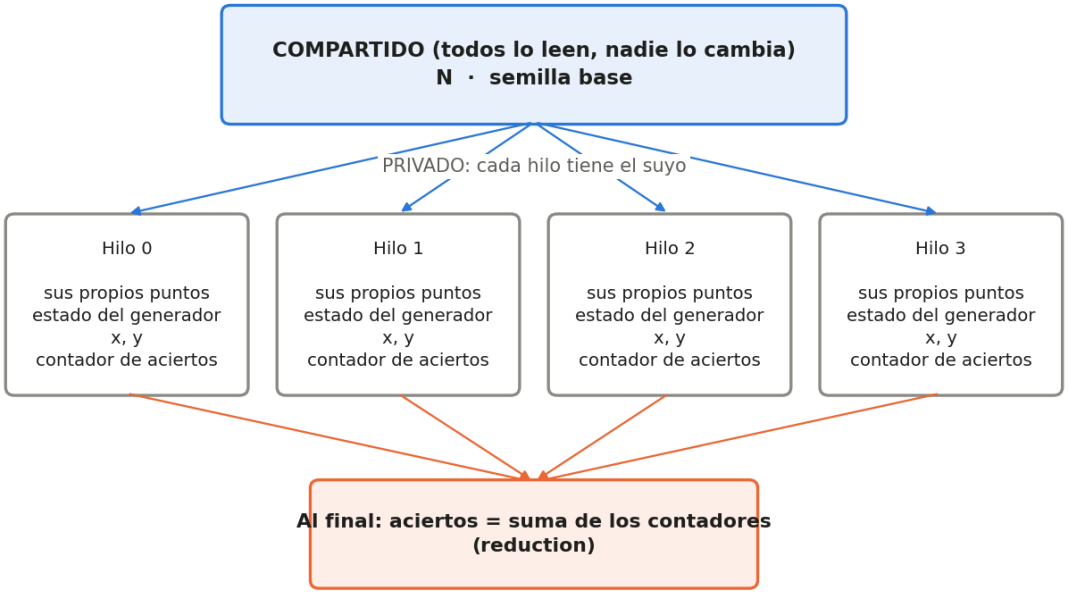


Figura I-3. Qué ven todos los hilos (arriba, compartido) y qué tiene cada uno (al medio, privado). Al final se suman los contadores.

I.3. Demuestre que repartir las N muestras entre p hilos y sumar los aciertos parciales de cada uno produce el mismo estimador que la versión secuencial. Con el mismo N total, ¿qué precisión estadística cabe esperar respecto de la ejecución con un solo hilo? Justifique.

Demostración. La versión secuencial cuenta los aciertos uno por uno:

$$\text{dentro} = a(1) + a(2) + \ldots + a(N),$$

donde cada a(i) vale 1 si el punto i cayó dentro y 0 si no. Si se reparten los N puntos en p grupos sin repetir ni dejar ninguno por fuera, cada hilo k suma solo los de su grupo y obtiene un parcial d(k). Al sumar los parciales:

$$d(1) + d(2) + \ldots + d(p) = a(1) + a(2) + \ldots + a(N) = \text{dentro},$$

porque es **la misma suma, solo que agrupada de otra forma**: el orden en que se suman números enteros no cambia el resultado. Entonces el estimador $4 \cdot \text{dentro} / N$ es **exactamente el mismo**. Como el contador es un número entero, no hay los pequeños errores de redondeo que aparecen al sumar decimales en otro orden (el caso que vimos en clase con reduction en punto flotante).

La condición es que cada hilo use **puntos distintos** y que entre todos sumen exactamente los N del programa secuencial. Lo comprobé en mi computador con $N = 10^8$ partido en 4 tramos (Figuras I-4 y I-5): 19 636 256 + 19 634 338 + 19 636 316 + 19 634 797 = **78 541 707**, el mismo conteo que el programa secuencial, y por tanto el mismo $\pi = 3{,}141668$.

¿Qué precisión se espera con p hilos? La misma que con un solo hilo. La precisión depende de **cuántos puntos se usan (N)**, no de cuántos hilos los calculan. Con $N = 10^8$ el error típico es

0,000164, sea con 1 hilo o con 8. Lo único que cambia al usar más hilos es el **tiempo**. Si se quiere más precisión, hay que aumentar N.

Esto vale solo si los hilos usan puntos distintos. Si todos usaran la misma semilla, repetirían los mismos puntos: en realidad habría solo N/p puntos diferentes y el error sería √p veces mayor. Ese error se analiza en la Parte II.

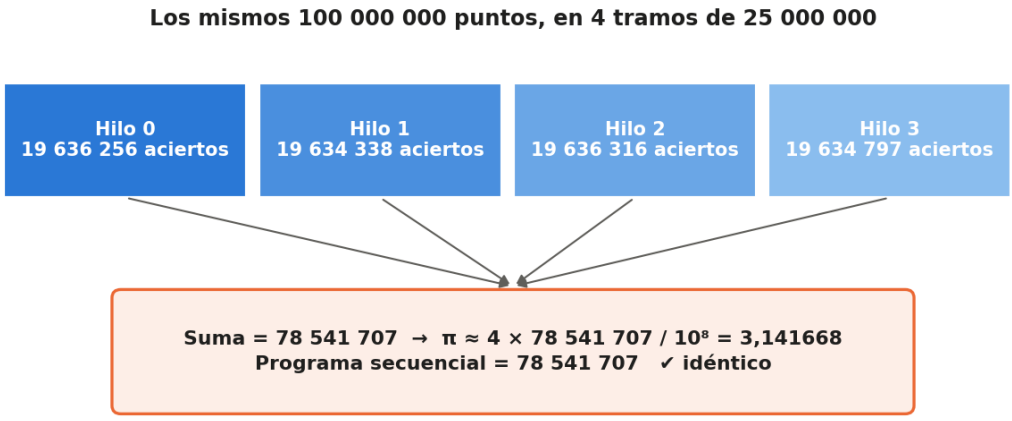


Figura I-4. Los 100 millones de puntos repartidos en 4 tramos: la suma de los aciertos de cada tramo es igual al conteo del programa secuencial.

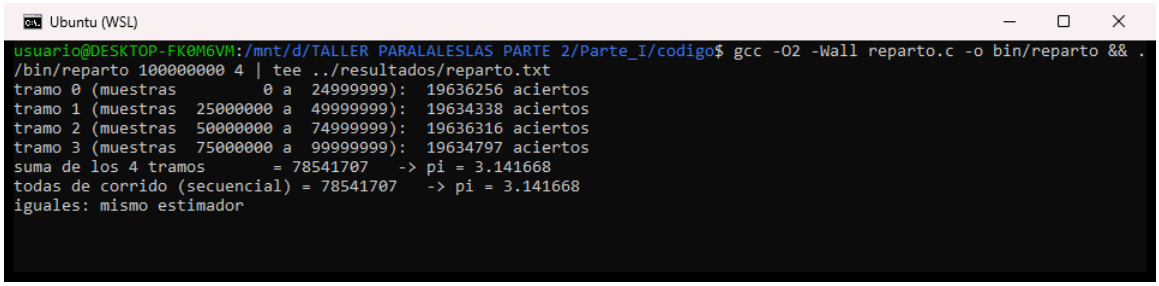


Figura I-5. Ejecución real en mi computador: aciertos de cada tramo, su suma y el conteo secuencial (son iguales).

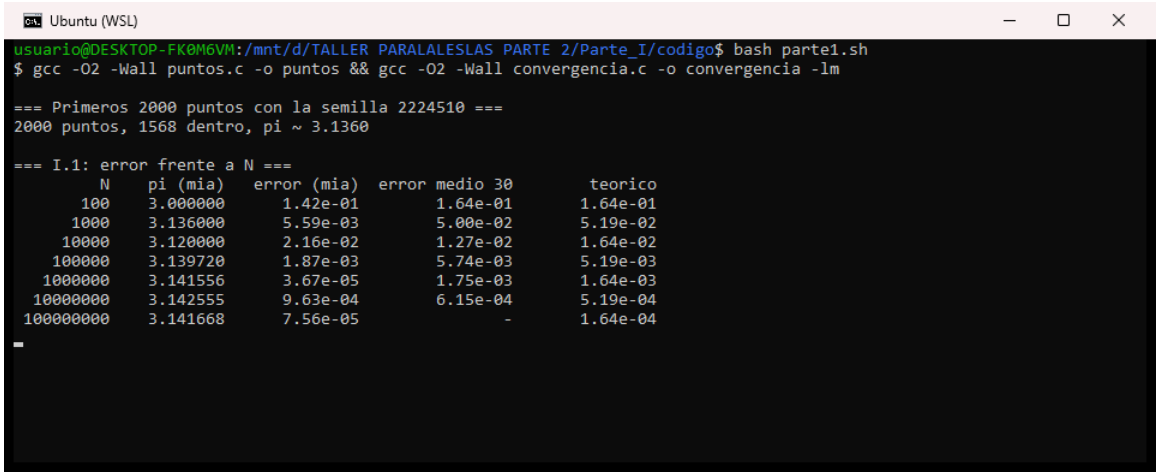


Figura I-6. Ejecución real de los experimentos de apoyo: los 2000 puntos de la Figura I-1 y la tabla de error frente a N de la Figura I-2.

Parte II. Implementación en OpenMP

Todo se ejecutó en mi portátil (AMD Ryzen 5 3450U: 4 núcleos físicos y 8 hilos lógicos; Ubuntu en WSL; gcc 15.2.0), conectado al cargador, con $N = 10^8$. El enunciado trae la semilla 20261018, pero el profesor indicó que la semilla de cada estudiante es su código, así que usé **2224510** en todos los programas.

II.1 Compile y ejecute la versión secuencial con $N = 10^8$. Verifique que la estimación es consistente con el valor de π (un error del orden de 10^{-4} es normal; explique de dónde sale ese orden de magnitud). Registre el tiempo de ejecución T(1) con omp_get_wtime() y consérvelo para la parte III.

Compilé el programa del enunciado con el comando indicado (gcc -O2 montecarlo.c -o montecarlo -lm), cambiando solo la semilla. Imprimió **$\pi \approx 3{,}141668$**: 78 541 707 de los 10^8 puntos cayeron dentro del círculo. El valor real es 3,141593, así que el error es de **0,0000756** ($7{,}56\times10^{-5}$).

¿De dónde sale el orden de 10^{-4}? Como expliqué en la Parte I, el error típico es $1{,}64/\sqrt{N}$. Con $N = 10^8$ queda $\sqrt{N} = 10\,000$, así que el error esperado es $1{,}64/10\,000 \approx$ **0,000164**, que es del orden de 10^{-4}. Mi error (0,0000756) es menor que eso, así que la estimación es consistente.

Tiempo: hice una copia del programa que mide el ciclo con omp_get_wtime(), sin cambiar nada más, y la corrí 7 veces. Los tiempos fueron de 1,1999 s a 1,2743 s, y la mediana es **T(1) = 1,2288 s** (Figura II-2). En la Parte III repito esta medida en la misma sesión que los tiempos con varios hilos, para que la comparación sea justa.

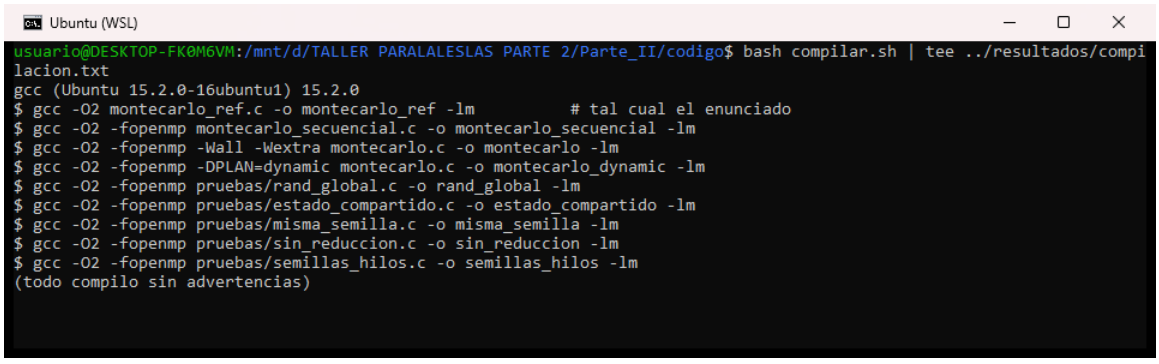


Figura II-1. Compilación del programa del enunciado, de la versión paralela (sin advertencias con -Wall -Wextra) y de los programas de prueba.

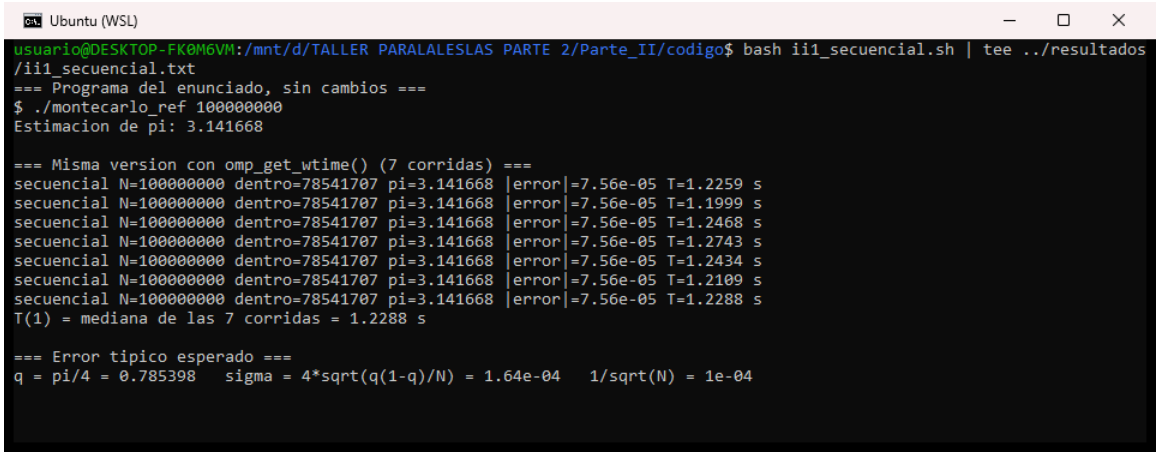


Figura II-2. II.1: el programa del enunciado da π = 3,141668; la versión con omp_get_wtime() da el mismo conteo en las 7 corridas, con mediana T(1) = 1,2288 s.

II.2 Implemente la versión paralela con OpenMP. Su programa debe:

- **usar una directiva omp parallel for con la cláusula de reducción adecuada sobre el acumulador de aciertos;**
- **permitir elegir el número de hilos p (con omp_set_num_threads o OMP_NUM_THREADS);**
- **garantizar que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento con el de los demás. Como semilla base use la que su docente le asignó (las semillas se publican por estudiante) y espacie las de cada hilo al menos N/p valores. El resultado debe ser reproducible: con las mismas semillas, el programa debe producir la misma estimación cualquiera que sea el valor de p.**

Mantuve el programa original casi igual: el mismo rand_r, la misma prueba $x^2 + y^2 \leq 1$ y el mismo ciclo. Lo único nuevo es que los 10^8 puntos se dividen en **1000 bloques de 100 000** y OpenMP reparte los bloques entre los hilos. El programa completo es montecarlo.c (anexo del documento):

```
#pragma omp parallel for reduction(+:dentro) schedule(static)
for (long b = 0; b < nbloques; b++) {
    unsigned int estado = saltar(semilla, b * BLOQUE * 6);
    /* aqui va el mismo ciclo del programa original */
}
```

Así cumple cada requisito:

- **Directiva y reducción:** omp parallel for con reduction(+:dentro). Cada hilo cuenta en su propia copia y al final se suman.
- **Elegir los hilos:** como segundo argumento (./montecarlo 100000000 4, que llama a omp_set_num_threads) o con OMP_NUM_THREADS=4 ./montecarlo 100000000. Las dos formas aparecen en la Figura II-4.
- **Flujos sin solapamiento y semillas espaciadas:** los números aleatorios salen de **una sola lista fija** que empieza en la semilla 2224510. La función saltar calcula en pocos pasos en

qué estado estaría el generador en cualquier punto de la lista, así que cada bloque arranca justo en "su" parte (Figura II-3). Es como repartir un libro por capítulos: cada hilo lee páginas distintas. Con 4 hilos, el hilo 0 empieza en la muestra 0, el 1 en la 25 000 000, el 2 en la 50 000 000 y el 3 en la 75 000 000, es decir, **separados N/p = 25 000 000 muestras**. Como cada muestra usa 6 valores del generador, la separación es de 150 millones de valores, mucho más que el mínimo de N/p que pide el enunciado. Comprobé que el estado inicial de cada hilo calculado con saltar es igual al que se obtiene avanzando rand_r uno por uno (columna "ok" de la Figura II-4).

- **Mismo resultado con cualquier p:** como cada bloque usa siempre los mismos números, sin importar qué hilo lo haga, el conteo fue **78 541 707 con 1, 2, 4 y 8 hilos**, en todas las repeticiones, y es el mismo del programa secuencial. Con 8 hilos el tiempo bajó de 1,33 s a 0,22 s.
- **Semilla:** está en SEMILLA_BASE (2224510) y también se puede pasar como tercer argumento.

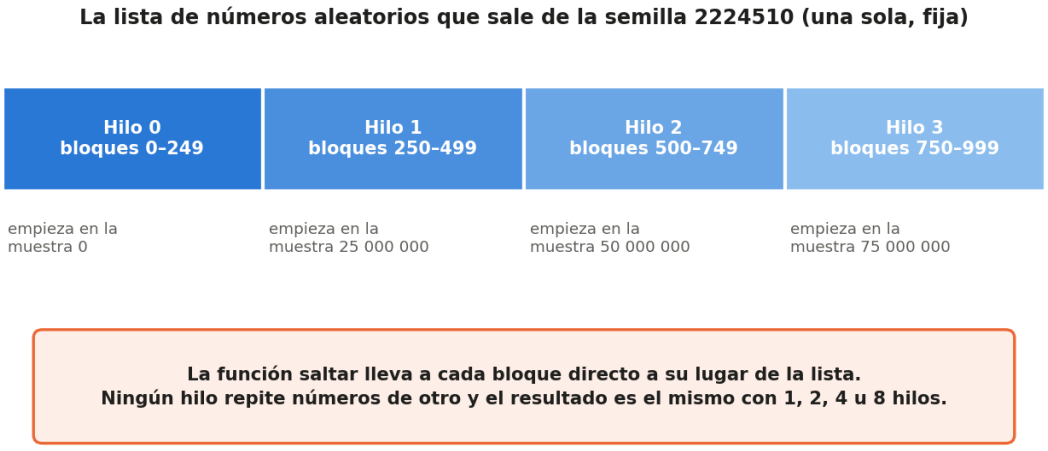


Figura II-3. Cómo se reparte el trabajo con 4 hilos: cada uno toma un tramo distinto de la misma lista de números aleatorios.

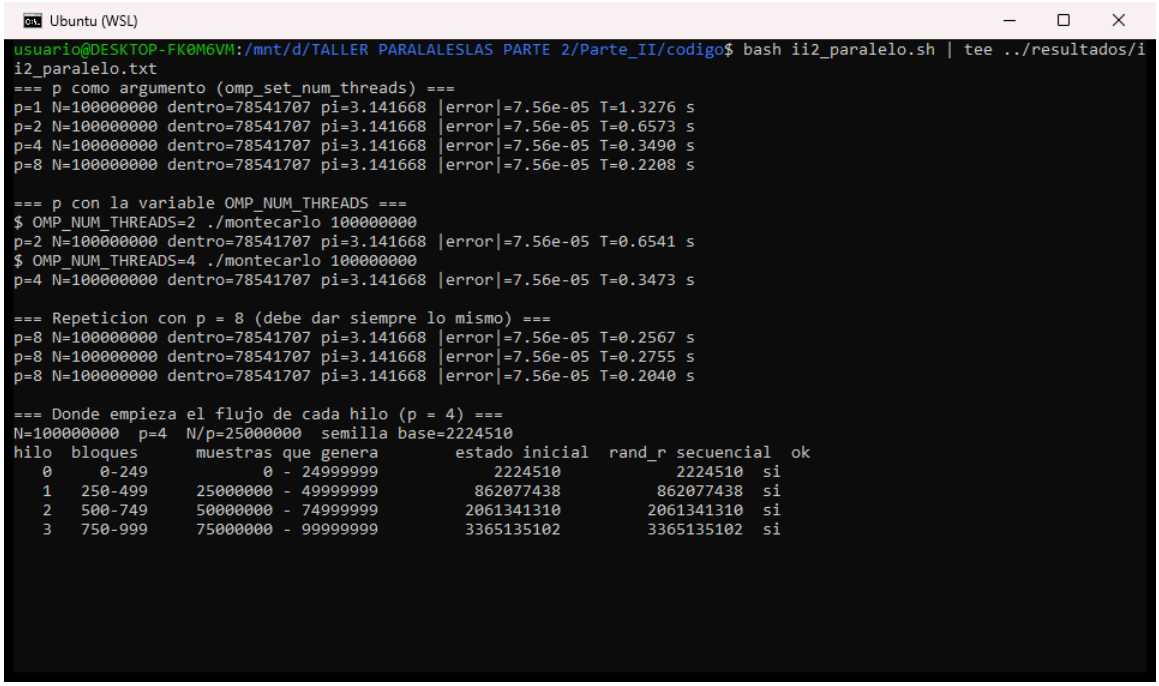


Figura II-4. II.2: el mismo conteo con 1, 2, 4 y 8 hilos (por argumento y con OMP_NUM_THREADS) y en tres repeticiones; abajo, la muestra donde empieza cada hilo y la verificación de su estado inicial.

II.3 Explique por qué no es correcto que todos los hilos llamen a la función rand() compartiendo el estado global del generador. Su respuesta debe cubrir al menos dos problemas distintos: uno de condición de carrera sobre el estado interno, y otro de correlación entre secuencias si cada hilo inicializara el generador con la misma semilla. Indique qué consecuencias tendría cada uno sobre la estimación de π y sobre el error reportado.

rand() guarda su posición en la lista en **una sola variable escondida**, que comparten todos los hilos. Para mostrar los dos problemas escribí programas que los cometen a propósito (Figuras II-5 y II-6).

Problema 1: condición de carrera sobre el estado. Es como si 8 personas escribieran a la vez en el mismo cuaderno: se pisan. Dos hilos pueden leer el mismo estado, sacar el mismo número o borrar el avance del otro.

- **Con rand() de verdad:** la biblioteca le pone un candado para que no se dañe, pero eso obliga a los hilos a turnarse. Con 8 hilos tardó **entre 11,2 s y 12,1 s, contra 1,56 s con un solo hilo**. Además, el conteo cambió en cada corrida (78 540 004, 78 539 170 y 78 538 732), porque depende de qué hilo llega primero.
- **Con un estado compartido sin candado** (lo que pasa con rand() en bibliotecas sin protección): el conteo también cambió en cada corrida (78 540 043, 78 529 433 y 78 531 926), y con 8 hilos fue más lento que con uno (2,2 s contra 1,3 s).

- **Consecuencia sobre π:** el resultado deja de ser reproducible y puede quedar sesgado, porque se repiten unos números y se saltan otros.
- **Consecuencia sobre el error reportado:** la fórmula 0,000164 supone 10^8 puntos independientes. Si hay números repetidos, en realidad hay menos puntos distintos, y el error verdadero es mayor que el que se reporta. En una corrida el error fue 0,000415, más del doble de lo esperado.

Problema 2: correlación por usar la misma semilla. Si cada hilo tuviera su propio generador pero todos empezaran con la misma semilla, **todos generarían exactamente los mismos puntos**.

- Lo comprobé: con la misma semilla, el resultado con 2, 4 y 8 hilos fue **idéntico, dígito por dígito, al del programa secuencial con solo N/2, N/4 y N/8 puntos**. Con 8 hilos dio π = 3,142351, lo mismo que 12 500 000 puntos.
- **Consecuencia sobre π:** el resultado cambia según p, y en realidad solo hay N/p puntos distintos. Repetir los mismos puntos 8 veces no aporta nada.
- **Consecuencia sobre el error reportado:** el error real crece como √p. Con 8 hilos el error real es el de 12,5 millones de puntos (0,000464 esperado, 0,000759 medido), pero si se reporta con la fórmula para 10^8 puntos se diría ±0,000164: casi 3 veces más precisión de la que hay de verdad.

Mi versión evita los dos problemas: cada hilo tiene su propio estado (sin carrera) y cada bloque empieza en un lugar distinto de la lista (sin correlación).

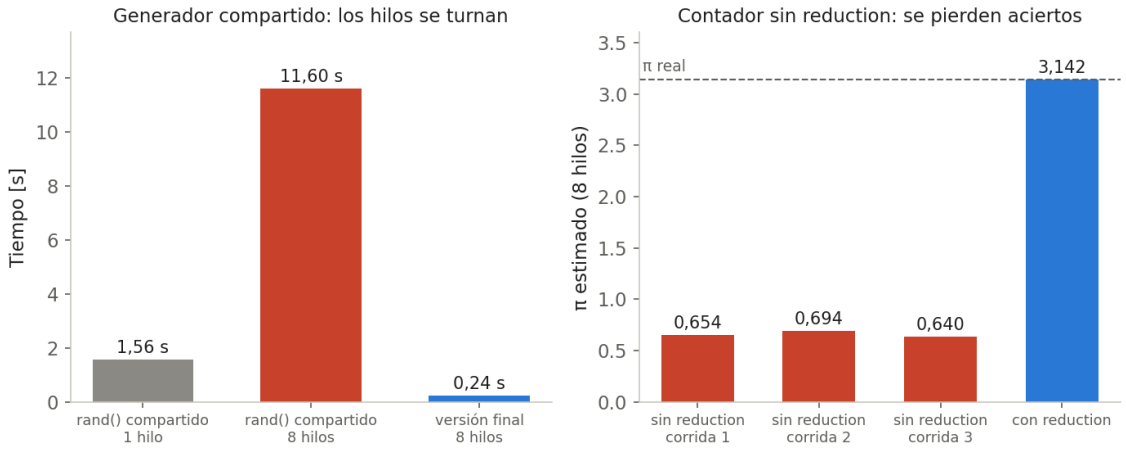


Figura II-5. Izquierda: con rand() compartido, 8 hilos tardan mucho más que uno. Derecha: sin reduction, π sale muy por debajo del valor real (ver II.4).

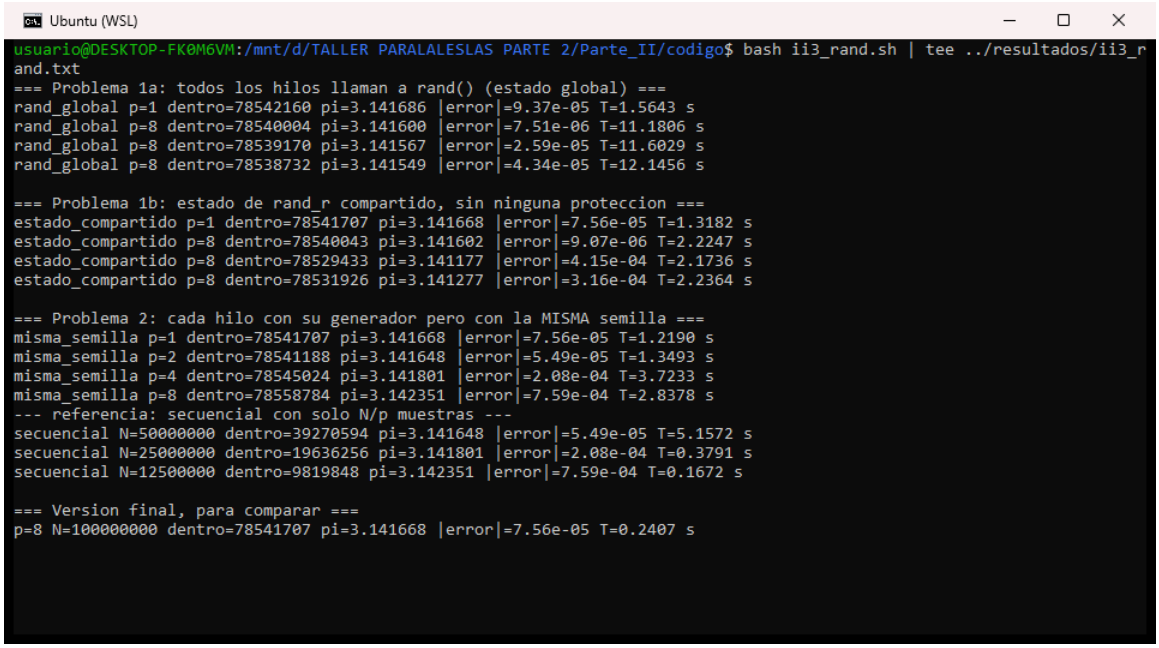


Figura II-6. II.3: rand() compartido y estado compartido (el conteo cambia en cada corrida); misma semilla en todos los hilos (el resultado es igual al de N/p puntos).

II.4 Analice los atributos de compartición de datos de su región paralela: enumere las variables que OpenMP trata como compartidas y como privadas por defecto en su código, explique la cláusula que usó para el acumulador y describa qué ocurriría si el acumulador se actualizara dentro del ciclo sin esa cláusula (diagnostique el tipo de defecto y su efecto sobre el resultado).

Una variable **compartida** es la misma para todos los hilos, y una **privada** es una copia propia de cada hilo. Como vimos en clase, lo que se declara fuera de la región paralela es compartido y lo que se declara dentro, junto con la variable del for, es privado. Para comprobarlo con mi código, compilé la región con default(none), que obliga a declarar todo. El compilador dio un error por cada variable compartida: **nbloques, semilla y N** (Figura II-8).

Variable	Tipo por defecto	Por qué
N, nbloques, semilla	Compartidas	Se declaran antes del ciclo y solo se leen
b	Privada	Es la variable del omp parallel for
estado, fin, i, x, y	Privadas	Se declaran dentro del ciclo
dentro	Compartida, pero reduction la vuelve privada	Es el contador de aciertos

Que N, nbloques y semilla sean compartidas no causa problemas, porque nadie las modifica.

Cláusula del acumulador: reduction(+:dentro). Cada hilo recibe una copia de dentro que empieza en 0, cuenta ahí sus aciertos sin estorbar a nadie y, al final, OpenMP suma todas las copias. Es la

opción más rápida, porque los hilos se coordinan una sola vez al final y no en cada acierto (en clase vimos que critical o atomic en cada acierto funcionan, pero son mucho más lentos).

Sin la cláusula: dentro sería una sola variable compartida, y dentro++ en realidad son tres pasos: leer, sumar 1 y guardar. Si dos hilos leen el mismo valor al mismo tiempo, los dos guardan el mismo resultado y se pierde un acierto (Figura II-7). Es una **condición de carrera** con actualizaciones perdidas. Lo probé quitando solo la cláusula (Figura II-8):

- con 1 hilo da bien (78 541 707), y por eso el error no se nota si solo se prueba sin paralelismo;
- con 8 hilos se perdió casi el 80 % de los aciertos: π salió **0,654; 0,694 y 0,640**, distinto en cada corrida;
- el efecto es un π siempre **menor** que el real (solo se pierden aciertos, nunca se agregan) y que cambia en cada ejecución.

Tiempo	Hilo 0	Hilo 1	dentro en memoria
1	lee dentro (100)		100
2		lee dentro (100)	100
3	suma 1 → 101	suma 1 → 101	100
4	escribe 101		101
5		escribe 101	**101 ← se perdió un acierto**

Figura II-7. Cómo se pierde un acierto cuando dos hilos actualizan el mismo contador al mismo tiempo.

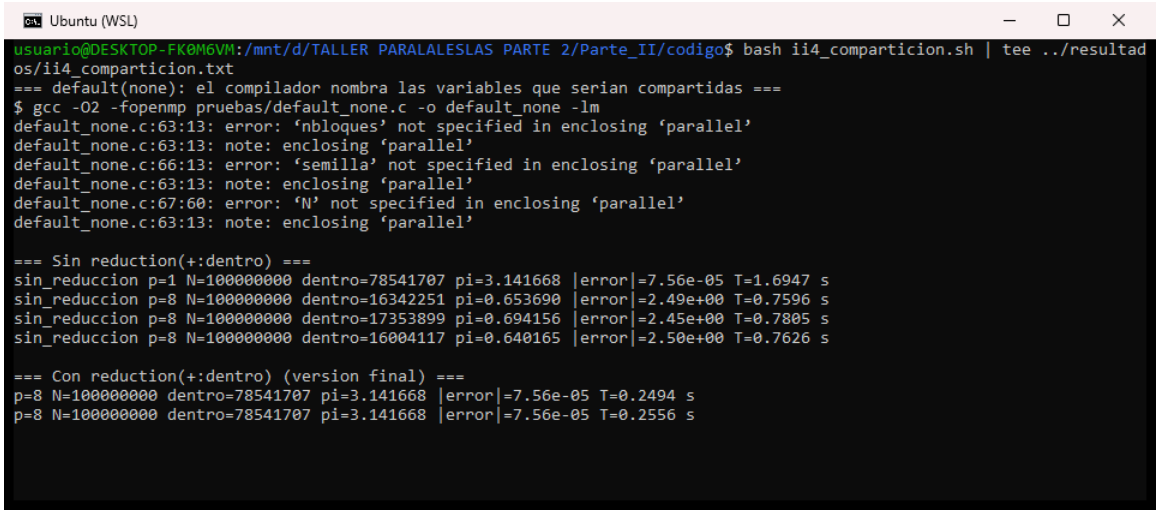


Figura II-8. II.4: con default(none) el compilador nombra las variables compartidas (nbloques, semilla y N); sin reduction π sale entre 0,64 y 0,69; con reduction sale bien.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

El desempeño de un programa paralelo se estudia con dos magnitudes. El *speedup* con p hilos es

$$S(p) = T(1) / T(p),$$

donde T(p) es el tiempo de ejecución con p hilos, y la *eficiencia* es E(p) = S(p)/p. Si una fracción f del trabajo total es paralelizable ($0 \leq f \leq 1$) y el resto es inevitablemente secuencial, la *ley de Amdahl* establece el speedup máximo alcanzable:

$$S(p) = 1 / [\ (1 - f) + f/p\ ].$$

Protocolo experimental

Con $N = 10^8$ fijo, mida T(p) para $p \in \{1, 2, 4, 8\}$ (ajuste el conjunto según los núcleos disponibles y las indicaciones de su docente). Para cada valor de p realice al menos cinco ejecuciones y registre la *mediana* de los tiempos; justifique por qué la mediana es más apropiada que el promedio en este experimento.

Equipo: AMD Ryzen 5 3450U con **4 núcleos físicos y 2 hilos por núcleo (8 hilos lógicos)**, según lscpu (Figura III-1), conectado al cargador. **Cómo medí:** hice 7 rondas. En cada ronda corrí el programa secuencial y luego el paralelo con 1, 2, 4 y 8 hilos, uno tras otro, para que todos sufrieran las mismas condiciones del equipo. T(1) es el programa secuencial (Figura III-2).

p (hilos)	Mediana de T(p) [s]	S(p) = T(1)/T(p)	E(p) = S(p)/p
1	1,2549	1,000	1,000
2	0,6826	1,838	0,919
4	0,3700	3,392	0,848
8	0,2499	5,022	0,628

Tabla III-1. Mediana de 7 ejecuciones con $N = 10^8$. Las 35 ejecuciones dieron el mismo conteo: 78 541 707 aciertos.

En la Parte II medí T(1) = 1,2288 s. Aquí usé el T(1) medido junto con los demás tiempos (1,2549 s), para comparar tiempos tomados en las mismas condiciones. Entre los dos hay solo un 2 % de diferencia.

¿Por qué la mediana y no el promedio? Porque a veces Windows u otro programa interrumpe una ejecución y la vuelve más lenta, y eso solo puede hacerla más lenta, nunca más rápida. En mis datos pasó: en la ronda 4, una corrida secuencial tardó **12,41 s** cuando lo normal era 1,25 s (Figura III-3). Con el promedio, T(1) sería 2,84 s y S(4) saldría 2,84/0,369 = 7,7, es decir, ¡más rápido que con 4 núcleos ideales!, algo imposible. La mediana toma el valor del medio, no le afectan esos casos raros y da T(1) = 1,25 s.

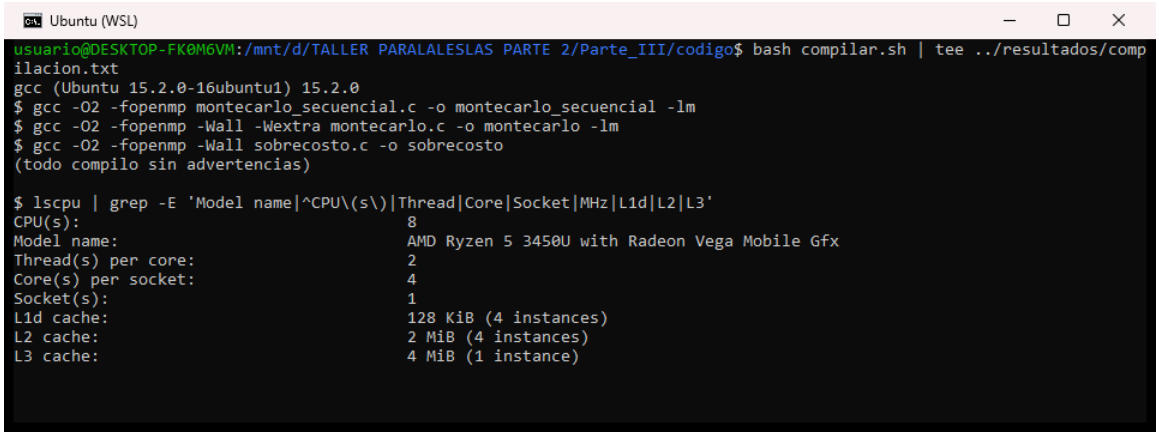


Figura III-1. Compilación sin advertencias y datos del procesador con lscpu: 4 núcleos (Core(s) per socket) y 2 hilos por núcleo (Thread(s) per core).

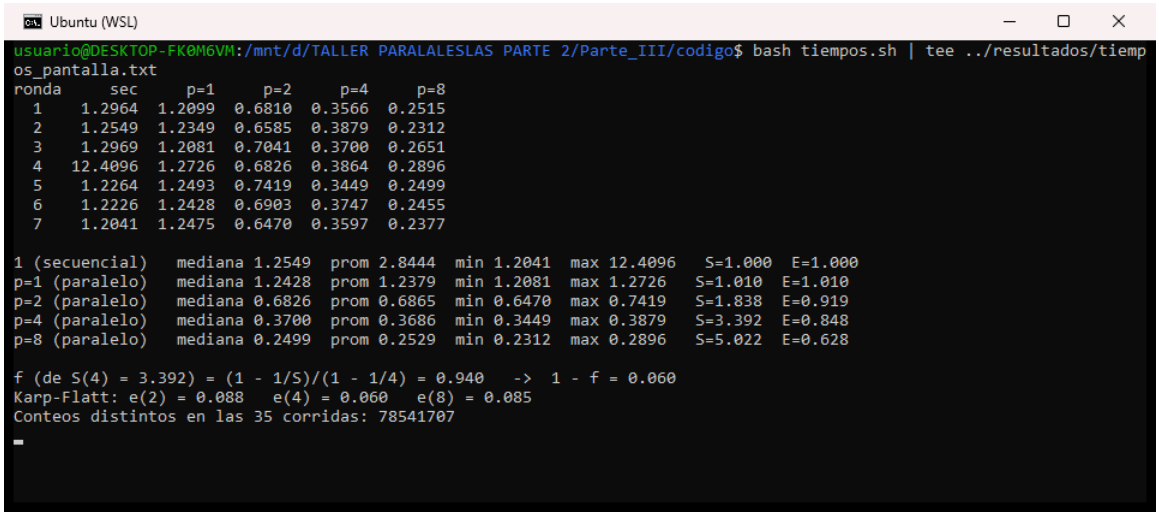


Figura III-2. Las 7 rondas de medición, las medianas, S(p), E(p) y los cálculos de f y de Karp–Flatt.

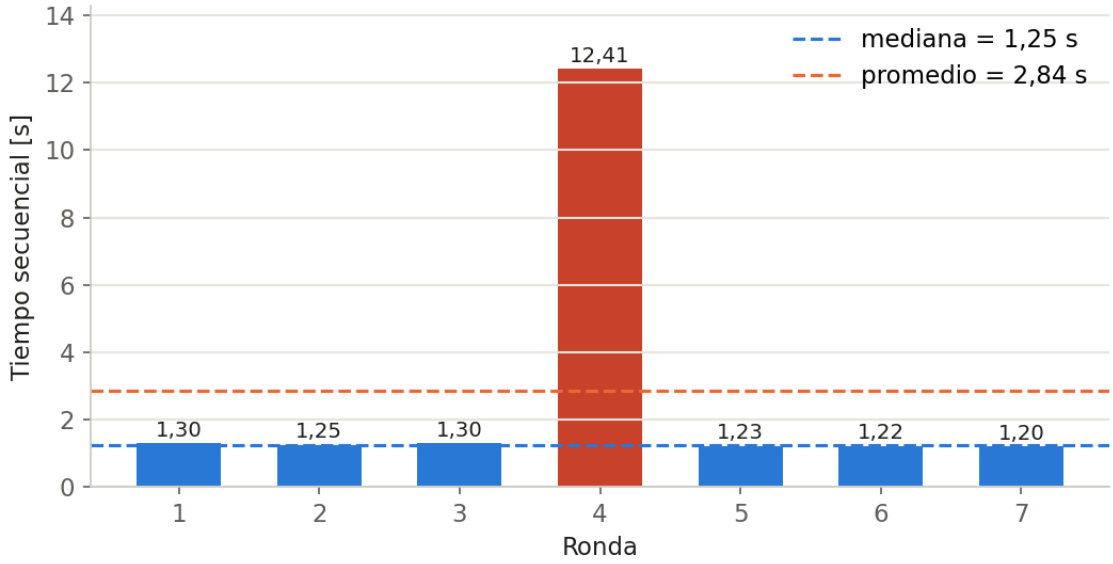


Figura III-3. Tiempo del programa secuencial en cada ronda. Una sola corrida lenta (ronda 4) dispara el promedio, pero no mueve la mediana.

Preguntas

III.1 Aplique la ley de Amdahl a los siguientes casos y interprete cada resultado: (a) f = 0,98, p = 8; (b) f = 0,90, p = 8; (c) f = 0,98 y p creciendo sin límite. ¿Qué le enseña el caso (c) sobre la fracción secuencial de un programa?

Caso	Cálculo	Speedup	Qué significa
(a) f = 0,98, p = 8	1 / (0,02 + 0,98/8) = 1 / 0,1425	**7,02**	Con solo un 2 % secuencial, 8 núcleos rinden como 7: casi ideal (E = 0,88)
(b) f = 0,90, p = 8	1 / (0,10 + 0,90/8) = 1 / 0,2125	**4,71**	Con un 10 % secuencial se desperdicia casi la mitad de los núcleos (E = 0,59)
(c) f = 0,98, p → ∞	1 / (0,02 + 0) = 1 / 0,02	**50**	Ni con infinitos núcleos se pasa de 50 veces más rápido

Lo que enseña el caso (c): la parte secuencial pone un **techo**. Por más núcleos que se agreguen, el programa nunca va más de 1/(1 − f) veces más rápido (Figura III-4). Con apenas un 2 % secuencial el techo es 50, y con un 10 % baja a 10. Por eso, para acelerar un programa, **reducir la parte secuencial vale más que comprar más núcleos**.

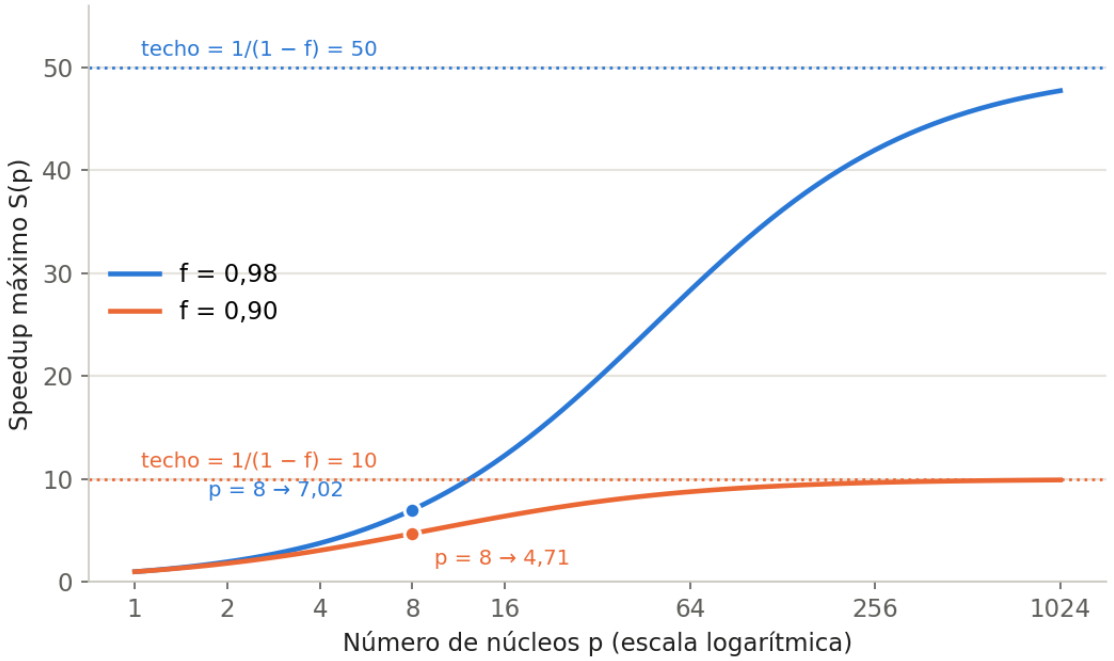


Figura III-4. Ley de Amdahl para f = 0,98 y f = 0,90. Las curvas se aplanan contra su techo 1/(1 − f) aunque p siga creciendo.

III.2 Con sus mediciones, estime la fracción secuencial f de su programa: despeje f de la ley de Amdahl a partir del speedup medido con p = 4 y muestre el procedimiento completo. Si lo desea, contraste su resultado con la métrica de Karp–Flatt, e = (1/S(p) − 1/p) / (1 − 1/p), y explique qué mide.

Partí de S(4) = 3,392 (Tabla III-1):

$$1/S = (1 - f) + f/4$$

$$1/S = 1 - f \cdot (1 - 1/4) = 1 - 0{,}75 \cdot f$$

$$f = (1 - 1/S) / 0{,}75$$

$$f = (1 - 1/3{,}392) / 0{,}75 = (1 - 0{,}2948) / 0{,}75 = 0{,}7052 / 0{,}75 = \mathbf{0{,}940}$$

O sea, el **94 %** del programa se comporta como paralelizable y la **fracción secuencial es 1 − f = 0,060** (6 %).

Karp–Flatt: la calculé con los tres valores de p (Figura III-2):

p	2	4	8
e(p)	0,088	0,060	0,085

Con p = 4 da 0,060, igual a 1 − f, porque es la misma cuenta. **Karp–Flatt mide la fracción secuencial "aparente", calculada a partir de lo que realmente se midió.** Lo útil es ver si cambia con p: si fuera siempre igual, habría de verdad un 6 % de código imposible de repartir. En mi caso

no es constante (0,088, 0,060, 0,085), lo que indica que el freno no es un pedazo fijo de código secuencial, sino costos y ruidos que cambian según el número de hilos.

Eso cuadra con mi programa. Lo único secuencial es preparar las variables e imprimir, y crear los 4 hilos cuesta 0,57 milisegundos de un total de 370 (0,15 %, Figura III-6). Ese 6 % "secuencial" viene en realidad del hardware y del sistema, como explico en III.3.

III.3 Compare su speedup medido con el ideal lineal S(p) = p y con el predicho por la ley de Amdahl usando su f estimado. Explique las discrepancias que observe. Considere en su análisis, entre otros factores, la creación y destrucción de hilos, el costo de la reducción, los límites de ancho de banda de memoria y la jerarquía de caché, y el efecto de la frecuencia variable del procesador y del hiperprocesamiento.

p	Ideal	Amdahl (f = 0,940)	Medido
2	2	1,89	1,84
4	4	3,39	3,39
8	8	5,64	5,02

Tabla III-2. Speedup ideal, predicho por Amdahl y medido.

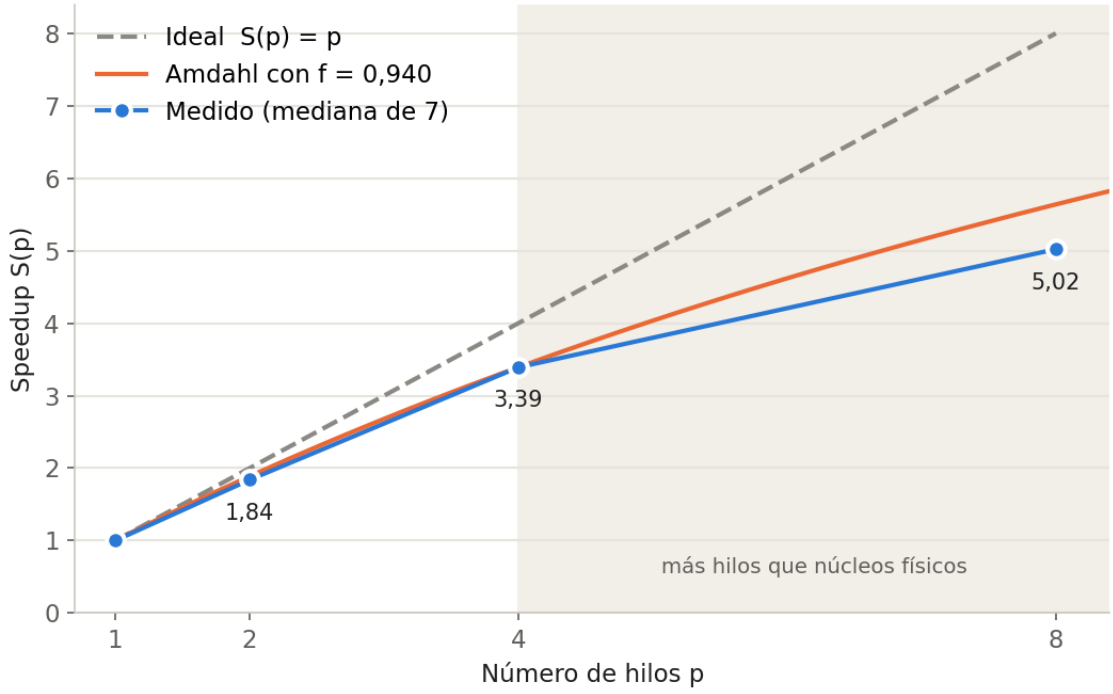


Figura III-5. Speedup medido (azul) frente al ideal (gris) y a Amdahl (naranja). La zona sombreada tiene más hilos que núcleos físicos.

- **Frente al ideal:** la distancia crece con p. Con 8 hilos el programa va 5,02 veces más rápido, no 8.

- **Frente a Amdahl:** coincide con p = 4 porque de ahí saqué f. Con 8 hilos medí bastante menos de lo que predice (5,02 contra 5,64). Amdahl supone un freno fijo, y en mi equipo el freno crece cuando hay más hilos que núcleos.

Revisé cada factor con mediciones en mi computador:

- **Crear y destruir hilos** (Figura III-6): crear los hilos cuesta 0,36 ms con 2, 0,57 ms con 4 y **13,6 ms con 8**. Con 4 hilos es solo el 0,15 % de los 370 ms del cálculo, pero con 8 hilos ya es **el 5,4 %** de los 250 ms. Es una de las razones por las que con 8 hilos se queda por debajo de Amdahl.
- **Costo de la reducción:** cada hilo suma su contador una sola vez al final. Medí entre 1 y 2,3 microsegundos: es despreciable.
- **Memoria y caché:** no influyen. El programa no usa arreglos: cada hilo trabaja con unas pocas variables (estado, x, y, contador) que caben en su caché L1 de 32 KiB. No hay tráfico de memoria que pueda saturarse.
- **Frecuencia variable** (Figura III-7): medí la velocidad del procesador con Windows mientras corría el programa. Con 1 hilo fue de unos **3010 MHz**, con 4 de **2912 MHz** y con 8 de **2572 MHz**. Cuando trabaja un solo núcleo, el procesador se acelera (turbo); con todos ocupados, baja. Con 4 hilos la baja es pequeña (3 %), pero con 8 es del **15 %**. Esta medición tiene ruido: la lectura con 2 hilos salió anómala (2239 MHz) y por eso no la uso. Lo que se repitió en mis intentos es que con 8 hilos la frecuencia fue menor que con 1 y con 4.
- **Hiperprocesamiento:** con 8 hilos hay 2 por núcleo, y cada par comparte el mismo núcleo físico. Por eso pasar de 4 a 8 hilos no duplicó la velocidad: la subió solo un 48 % (de 3,39 a 5,02), sumado a la baja de frecuencia y a los 13,6 ms de crear los hilos.
- **¿Y con 4 hilos?** La frecuencia (3 %) y la creación de hilos (0,15 %) no explican toda la distancia entre 3,39 y 4. Sospeché que con schedule(static) todos esperaban al hilo más lento, así que lo probé con schedule(dynamic), donde los hilos libres toman más trabajo (Figura III-8). **No mejoró** (0,456 s contra 0,412 s con static), así que descarté esa causa. El resto lo atribuyo a que Windows y Ubuntu (WSL) comparten los mismos 4 núcleos con mi programa: cualquier tarea del sistema le quita tiempo a algún hilo. No pude aislarlo con más precisión.

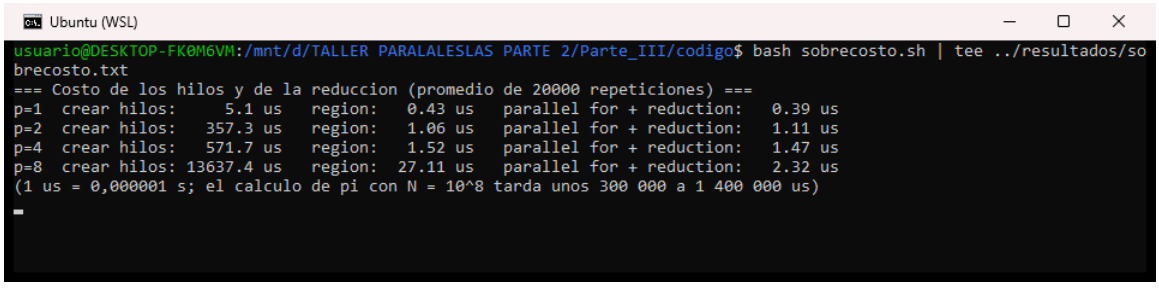


Figura III-6. Costo de crear los hilos, de una región paralela y de la reducción (1 us = una millonésima de segundo).

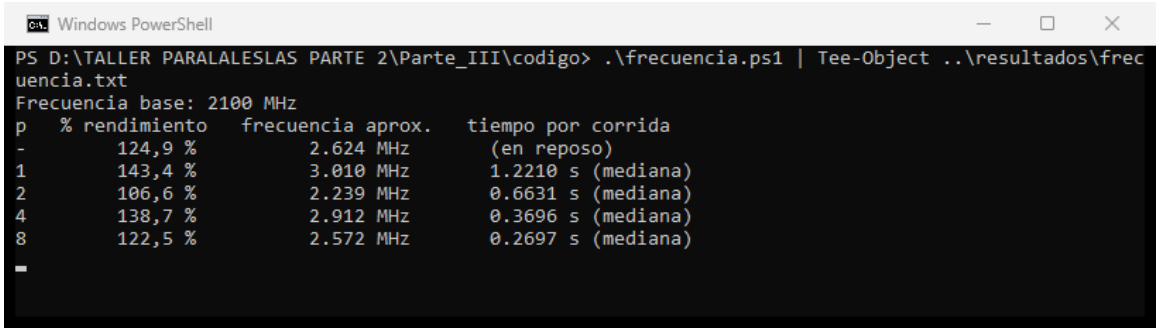


Figura III-7. Frecuencia del procesador medida con Windows mientras corría el programa con 1, 2, 4 y 8 hilos.

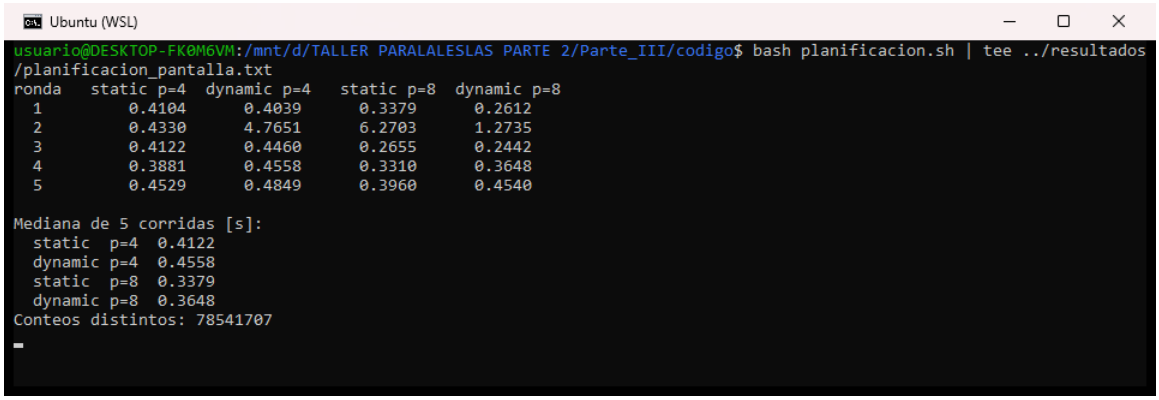


Figura III-8. Prueba de schedule(static) contra schedule(dynamic): dynamic no fue más rápido, así que el freno no era que unos hilos esperaran a otros.

III.4 Suponga que le ofrecen tiempo de cómputo en una máquina con 64 núcleos. Con su fracción f estimada, ¿qué speedup máximo podría obtener y en qué se diferenciaría de crecer el número de muestras N? Reflexione: aumentar p con N fijo, ¿mejora la precisión del resultado o solo el tiempo de cómputo? ¿Qué mejora cada una de las dos variables?

Con mi f = 0,940:

$$S(64) = 1 / (0{,}060 + 0{,}940/64) = 1 / (0{,}060 + 0{,}0147) = \mathbf{13{,}4}$$

El techo, con infinitos núcleos, es $1/0{,}060 \approx \mathbf{16{,}7}$. Es decir, 64 núcleos harían el programa solo unas 13 veces más rápido, con una eficiencia de apenas el 21 %. Hay que aclarar que mi f sale de un portátil con turbo, hiperprocesamiento y Windows de fondo. En un servidor de 64 núcleos esos efectos son distintos y el resultado podría ser mejor.

Aumentar p y aumentar N mejoran cosas distintas (Figura III-9):

- **Más hilos con el mismo N solo mejora el tiempo.** Con 1, 2, 4 y 8 hilos el resultado fue exactamente el mismo (78 541 707 aciertos, π = 3,141668) y el tiempo bajó de 1,30 s a 0,25 s. La precisión no cambia nada.

- **Más muestras mejora la precisión, pero cuesta tiempo.** El error esperado baja como $1/\sqrt{N}$: es 0,0016 con 10^6 puntos y 0,00008 con 4×10^8. Con 4×10^8 mi error fue de apenas 0,0000091, pero el tiempo subió de 0,27 s a 1,10 s. Para duplicar la precisión hacen falta 4 veces más puntos.

En resumen: p decide **qué tan rápido** sale el resultado y N decide **qué tan bueno** es. Lo ideal es combinarlos: con 64 núcleos conviene usar muchas más muestras en el mismo tiempo, y así ganar precisión sin esperar más (la idea de Gustafson). Mi programa tiene un límite para eso: rand_r repite su lista después de unos 7×10^8 puntos, así que para N mucho mayor habría que usar otro generador.

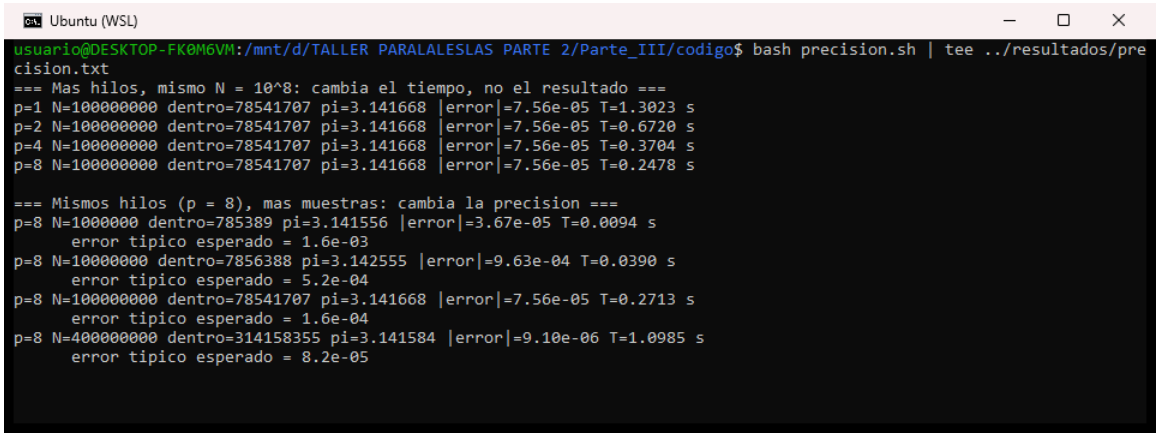


Figura III-9. Arriba: más hilos con el mismo N dan el mismo resultado en menos tiempo. Abajo: más muestras con 8 hilos dan menos error, pero tardan más.

Parte IV. Informe sobre su proceso de solución

1. Las decisiones de diseño que tomó y por qué las tomó: estructura del programa, cláusulas de OpenMP empleadas y manejo de las semillas de los generadores de cada hilo.

- **Estructura:** quise cambiar lo menos posible el programa del enunciado. Conservé el mismo ciclo, el mismo rand_r y la misma prueba $x^2 + y^2 \leq 1$. Lo único nuevo es que los 10^8 puntos se dividen en 1000 bloques de 100 000, y cada vuelta del omp parallel for hace un bloque completo.
- **Cláusulas:** reduction(+:dentro), para que cada hilo cuente en su propia copia y se sume una sola vez al final (en clase vimos que es más barato que critical o atomic); schedule(static), porque todos los bloques cuestan lo mismo; y el número de hilos como argumento o con OMP_NUM_THREADS.
- **Semillas:** el enunciado exige el mismo resultado con cualquier número de hilos. Si cada hilo tuviera una semilla propia separada N/p, los puntos cambiarían al cambiar p. Por eso hice que los números dependan **del bloque y no del hilo**: la función saltar lleva cada bloque directo a su lugar en la lista de números de rand_r. Así cada bloque usa siempre los mismos números, sin importar qué hilo lo ejecute.

2. Los problemas que aparecieron durante el desarrollo y cómo los diagnosticó y corrigió (por ejemplo: resultados distintos en cada ejecución, estimaciones sesgadas, condiciones de carrera) y qué evidencia le permitió identificar cada uno.

Problema	Cómo lo noté (evidencia)	Qué hice
Semilla equivocada	Usé la del enunciado (20261018); el profesor aclaró que es el código de cada uno	La cambié a 2224510 y repetí todo
Generador compartido (carrera)	Con rand() y 8 hilos el conteo cambió en cada corrida y tardó 11 s contra 1,6 s	Cada bloque usa su propio estado de rand_r
Misma semilla en todos los hilos	Con 8 hilos $\pi = 3{,}142351$, idéntico al de solo N/8 puntos	Cada bloque empieza en un lugar distinto con saltar
Contador sin reduction (carrera)	Con 1 hilo daba bien; con 8, $\pi = 0{,}654$, 0,694 y 0,640	reduction(+:dentro)
Semillas "distintas" que no lo eran	En la Parte I, las semillas $2224510 + 1000 \cdot k$ daban un error medio 2,7 veces menor que el teórico: sus listas se solapaban	Usé tramos sin solapamiento con saltar; el error coincidió con la fórmula
Mediciones con ruido	Una corrida de 12,4 s (lo normal era 1,25 s); una lectura de frecuencia de 544 MHz con el procesador ocupado	Usé la mediana y medí la frecuencia con muestras más largas

3. El método que usó para validar la correctitud de sus resultados, tanto la estimación de π como la reproducibilidad con distinto número de hilos.

- **π:** con $N = 10^8$ obtuve 3,141668, con un error de 0,0000756, menor que el error típico esperado (0,000164).
- **Reproducibilidad:** exigí que el **conteo fuera idéntico**, no solo "parecido". Dio 78 541 707 en el programa secuencial, con 1, 2, 4 y 8 hilos, con OMP_NUM_THREADS y en las 35 corridas de la Tabla III-1. Además comprobé que el estado inicial de cada hilo con saltar coincide con el de rand_r avanzando uno por uno.
- **Código limpio:** montecarlo.c compila sin advertencias con -Wall -Wextra.

4. La conexión explícita entre su informe y sus datos: cite valores concretos de su tabla (por ejemplo, por qué su mediana con p = 4 resultó como resultó) en lugar de descripciones genéricas que podrían aplicarse a cualquier entrega.

Con p = 4, los siete tiempos fueron de 0,3449 s a 0,3879 s, y la **mediana quedó en 0,3700 s**. Por eso S(4) = 1,2549/0,3700 = 3,39 y no 4. Crear los hilos cuesta solo 0,57 ms (0,15 %) y la frecuencia bajó apenas un 3 % (de 3010 a 2912 MHz). Probé schedule(dynamic) por si unos hilos esperaban a otros, y no mejoró (0,456 s contra 0,412 s). Así que el resto lo atribuyo a que Windows y WSL usan los mismos 4 núcleos que mi programa. Con p = 8, S(8) = 5,02, por debajo de lo que predice Amdahl con f = 0,940 (5,64). Ahí sí hay causas medidas: crear 8 hilos costó 13,6 ms (5,4 % del tiempo), la frecuencia bajó un 15 % y hay 2 hilos compartiendo cada núcleo físico. En la Tabla III-1 la mediana también fue clave: la corrida de 12,41 s habría hecho que el promedio diera un S(4) imposible de 7,7.

5. Una reflexión final sobre lo que aprendió y sobre lo que haría de manera diferente en un próximo trabajo paralelo.

Aprendí que la reproducibilidad hay que diseñarla desde el principio: atar los números aleatorios al bloque y no al hilo la dio para cualquier p. También aprendí que un error de concurrencia puede esconderse (sin reduction el programa acierta con un hilo) y que dos semillas distintas no garantizan números distintos. Y que medir en un portátil tiene ruido: hay que repetir, usar la mediana y no confiar en una sola lectura. **En un próximo trabajo** confirmaría primero los datos asignados (la semilla), usaría un generador hecho para trabajar en paralelo y sin el límite de 7×10^8 puntos de rand_r, y fijaría cada hilo a un núcleo (OMP_PROC_BIND) para separar mejor el efecto del hiperprocesamiento.

Anexo. Bitácora de IA

#	Herramienta	Propósito	Qué aportó	Qué acepté / descarté y por qué
1	Claude Code (Opus 5.5), 27/10/2026	Parte I	Programas de apoyo (puntos, error frente a N, reparto en tramos), figuras y borrador de las respuestas	Acepté apoyar cada respuesta con un experimento real. Descarté la primera versión del experimento de error, cuyas semillas 2224510 + 1000·k daban resultados correlacionados
2	Claude Code (Opus 5.5), 27/10/2026	Parte II	Programa paralelo con saltar, programas que cometen cada error a propósito, scripts, capturas y borrador	Acepté el diseño por bloques porque conserva rand_r y da el mismo resultado con cualquier p. Descarté la semilla del enunciado y usé mi código, 2224510
3	Claude Code (Opus 5.5), 27/10/2026	Parte III	Scripts de tiempos, costo de los hilos, frecuencia y planificación, gráficas y borrador	Acepté medir cada factor en vez de suponerlo. Descarté una lectura de frecuencia errónea (544 MHz) y la idea de que los hilos se esperaban, porque dynamic no mejoró
4	Claude Code (Opus 5.5), 27/10/2026	Parte IV	Borrador de este informe a partir de mis resultados	Acepté la estructura. Revisé que cada cifra coincida con mis salidas en las carpetas resultados/

Anexo. Programa paralelo completo (montecarlo.c)

Compila sin advertencias con gcc -O2 -fopenmp -Wall -Wextra montecarlo.c -o montecarlo -lm.

```
/* montecarlo.c -- version paralela con OpenMP (version final).
   Uso: ./montecarlo <numero de muestras> [hilos] [semilla]
        (el numero de hilos tambien puede fijarse con OMP_NUM_THREADS)
   Compilar: gcc -O2 -fopenmp montecarlo.c -o montecarlo -lm

   Idea: la secuencia de rand_r con la semilla base se parte en bloques
   consecutivos de BLOQUE muestras. El estado inicial de cada bloque se
   calcula "saltando" hacia adelante en el generador, asi que cada bloque
   consume siempre los mismos numeros, sin importar que hilo lo ejecute.
   Resultado: flujos disjuntos (sin solapamiento) y la misma estimacion
   para cualquier p, identica ademas a la de la version secuencial. */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define SEMILLA_BASE 2224510u /* semilla asignada: codigo 2224510 */
#define BLOQUE 100000L        /* muestras por bloque */

#ifndef PLAN
#define PLAN static           /* -DPLAN=dynamic para otra planificacion */
#endif

/* rand_r de glibc avanza su estado con 3 pasos del LCG
   s <- 1103515245*s + 12345 (mod 2^32) en cada llamada,
   y cada muestra (x, y) hace 2 llamadas: 6 pasos por muestra. */
#define LCG_A 1103515245u
#define LCG_C 12345u
#define PASOS_POR_MUESTRA 6ULL

/* Avanza el estado s en k pasos del LCG con O(log k) operaciones. */
static unsigned int saltar(unsigned int s, unsigned long long k) {
    unsigned int a = LCG_A, c = LCG_C;  /* transformacion de 2^j pasos */
    unsigned int A = 1u, C = 0u;        /* transformacion acumulada */
    while (k) {
        if (k & 1ULL) {
            A = a * A;
            C = a * C + c;
        }
        c = a * c + c;
        a = a * a;
        k >>= 1;
    }
    return A * s + C;
}

int main(int argc, char *argv[]) {
    long N = (argc > 1) ? atol(argv[1]) : 100000000L;
    if (argc > 2) omp_set_num_threads(atoi(argv[2]));
    unsigned int semilla = (argc > 3) ? (unsigned int)strtoul(argv[3], NULL, 10)
                                      : SEMILLA_BASE;

    /* El salto debe coincidir con lo que hace rand_r en esta libc. */
    unsigned int prueba = semilla;
    rand_r(&prueba);
    rand_r(&prueba);
```

```
    if (prueba != saltar(semilla, PASOS_POR_MUESTRA)) {
        fprintf(stderr, "error: rand_r de esta libc no usa el LCG esperado\n");
        return 1;
    }
    /* Periodo del estado: 2^32. Con mas de 2^32/6 muestras la secuencia
       se repetiria y los bloques dejarian de ser disjuntos. */
    if ((unsigned long long)N * PASOS_POR_MUESTRA > 4294967296ULL)
        fprintf(stderr, "aviso: N supera el periodo del generador\n");

    long nbloques = (N + BLOQUE - 1) / BLOQUE;
    long dentro = 0;

    double t0 = omp_get_wtime();
    #pragma omp parallel for reduction(+:dentro) schedule(PLAN)
    for (long b = 0; b < nbloques; b++) {
        unsigned int estado =
            saltar(semilla, (unsigned long long)b * BLOQUE * PASOS_POR_MUESTRA);
        long fin = (b + 1) * BLOQUE < N ? (b + 1) * BLOQUE : N;
        for (long i = b * BLOQUE; i < fin; i++) {
            double x = rand_r(&estado) / (double)RAND_MAX;
            double y = rand_r(&estado) / (double)RAND_MAX;
            if (x * x + y * y <= 1.0)
                dentro++;
        }
    }
    double t = omp_get_wtime() - t0;

    double pi = 4.0 * dentro / (double)N;
    printf("p=%d N=%ld dentro=%ld pi=%.6f |error|=%.2e T=%.4f s\n",
           omp_get_max_threads(), N, dentro, pi, fabs(pi - M_PI), t);
    return 0;
}
```